

ARE 212 SECTION 9: Non-Standard Standard Errors I

Fiona Burlig

March 17, 2016

Problem Set 2 retrospective

In general, people did a nice job with PS2! The one issue that seemed to trip a few people up was the F test. Recall that one easy formula to use when testing model restrictions is:

$$F = \frac{(SSR_{restr} - SSR_{unres})/J}{SSR_{unres}/(n - k)}$$

The F -test formula we used in Section 4 would work as well, but this is the easiest thing to do. A couple of people also got tripped up with the number of restrictions. $J = 8$ in this setting, because you have 4 restrictions on intercepts ($\beta_{I1} = \beta_{I2}, \beta_{I2} = \beta_{I3}, \beta_{I3} = \beta_{I4}$, and $\beta_{I4} = \beta_{I5}$ - you get one for free), and four restrictions on slopes: ($\beta_{S1} = \beta_{S2}, \beta_{S2} = \beta_{S3}, \beta_{S3} = \beta_{S4}$, and $\beta_{S4} = \beta_{S5}$). Otherwise, people did a pretty nice job. Two other minor comments: check out the .Rnw files for the section notes to see how to make your figures not take up 3/4 of a page; lots of you are still giving output in big list format. It'd be nice to see more of you starting to use `xtable()` or `stargazer()` to make prettier looking output tables - take a look at the code below for some details on using `xtable()`!

Off we go

This week, we'll walk through ways to calculate standard errors for linear and non-linear combinations of coefficients. We'll do this analytically, and then using the delta method. We'll also cover the distinction between confidence intervals and prediction intervals. Without further ado:

$2 + 2 = 5?$

We'll start by thinking about the standard error of a linear combination of coefficients. First, let's set up our usual environment, including the OLS function we used in week 5 that calculates standard errors, t statistics, and p -values on t -tests (don't worry about the F -test, we won't use that this week). We'll add a couple new packages. One quickly auto-generates L^AT_EX code for decent looking tables: `xtable`. The other two contain canned routines that will help us verify our standard error estimates: `gmodels` and `msm`.

```
install.packages('xtable')
install.packages('gmodels')
install.packages('msm')
```

```
##### PACKAGES
library(dplyr)
library(readr)
library(magrittr)
library(lfe)
library(ggplot2)
library(xtable)
library(gmodels)
library(msm)

##### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  se <- sqrt(s2 * diag(XpXinv))
  tStat <- (betahat - 0) / se

  pVal <- tStat %>%
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*" (2)

  # THIS IS NEW! FOR THE PURPOSES OF MAKING TABLES
  # NOTICE THAT WE CAN TURN THIS INTO A TBLDF BECAUSE
  # ALL THE OBJECTS ARE THE SAME SHAPE/SIZE BY CONSTRUCTION
  olsOut <- as.tbl_df(cbind(betahat, se, tStat, pVal))
  names(olsOut) <- c("betahat", "SE", "tStat", "pVal")
  return(olsOut)
}
```

```
##### RANDOMIZATION SEED
set.seed(12345)

##### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)

##### DATA
autos <- read_csv("autos.csv") %>%
  mutate(ones = 1)
```

Suppose we estimate the following model:

$$\text{price}_i = \alpha + \beta_1 \text{MPG}_i + \beta_2 \text{weight}_i + \varepsilon_i$$

We're pros at estimating that via OLS by now (but I'm letting us use a new variable today). Let's use the `xtable()` function to auto-generate a pretty table in L^AT_EX:

```
myOLS <- OLS(autos, "price", c("ones", "mpg", "weight"))
xtable(myOLS)

## % latex table generated in R 3.2.3 by xtable 1.8-2 package
## % Thu Mar 17 09:14:09 2016
## \begin{table}[ht]
## \centering
## \begin{tabular}{rrrrr}
## \hline
## & betahat & SE & tStat & pVal \\
## \hline
## 1 & 1946.07 & 3597.05 & 0.54 & 0.59 \\
## 2 & -49.51 & 86.16 & -0.57 & 0.57 \\
## 3 & 1.75 & 0.64 & 2.72 & 0.01 \\
## \hline
## \end{tabular}
## \end{table}
```

	betahat	SE	tStat	pVal
1	1946.07	3597.05	0.54	0.59
2	-49.51	86.16	-0.57	0.57
3	1.75	0.64	2.72	0.01

Cool. But what if we're not interested in the coefficients themselves, but rather the the average "effect" on price of having an MPG of 20 and weighing 3,000 pounds.¹ We can call this thing we'd like to estimate LC , for linear combination, where $LC = 20\beta_1 + 5\beta_2$. We can use $\hat{\beta}_1$ and $\hat{\beta}_2$ to estimate the mean of LC , which you can think of like a point estimate:

```
(LC <- 20 * myOLS$betahat[2] + 3000 * myOLS$betahat[3])
## [1] 4249.433
```

You should be suspicious of this point estimate, because it doesn't come with a standard error.² Computing the standard error of this estimate, however, isn't completely trivial. The good news is that econometricians smarter than us have figured this out already. We can do it in one of two ways.

Analytical

The most conceptually straightforward way of doing this is to calculate the standard errors analytically. Recall that we can write the standard error of an estimate, $se(b)$, as:

$$se(b) = \sqrt{\sigma^2 \hat{b}})$$

where $\hat{\sigma}^2$ is just the estimated variance of the estimate.³ Thankfully, we can compute this badboy using what we know about variances and covariances.⁴ In particular, we know that:

$$\mathbb{V}(X + Y) = \mathbb{V}(X) + \mathbb{V}(Y) + 2\text{Cov}(X, Y)$$

for any random variables X, Y . We also know that:

$$\mathbb{V}(aX) = a^2\mathbb{V}(X)$$

for a constant a and random variable X . Finally, we of course remember that:

$$\text{Cov}(aX, bY) = ab\text{Cov}(X, Y)$$

for constants a, b and random variables X, Y . Using these rules, we can compute the variance of LC as:

$$\begin{aligned}\mathbb{V}(LC) &= \mathbb{V}(20\beta_1) + \mathbb{V}(3000\beta_2) + 20 \times 3000 \times 2\text{Cov}(\beta_1, \beta_2) \\ &= 400\mathbb{V}(\beta_1) + 9000000\mathbb{V}(\beta_2) + 120000\text{Cov}(\beta_1, \beta_2)\end{aligned}$$

We can estimate $\mathbb{V}(\beta_1)$, $\mathbb{V}(\beta_2)$, and $\text{Cov}(\beta_1, \beta_2)$ using the variance-covariance matrix of our OLS coefficients, recalling that $\mathbb{V}(\hat{\beta}) = s^2(\mathbf{X}'\mathbf{X})^{-1}$. In our 3×3 variance-covariance matrix, these will be the objects in $[2,2]$, $[3,3]$, and $[2,3]$ ⁵, respectively. Let's write a function that returns this matrix:⁶

¹Notice "effect" in quotes. This isn't a class about causality, so we should be careful with what we say. Suspend your disbelief, and suppose that MPG and weight are randomly assigned for now.

²How often do you see estimates presented without standard errors in the real world? All the time. You should be suspicious all the time.

³How many times can *you* fit 'estimate' into a sentence?

⁴Or, in my case, what's living, covered in cobwebs, in some back corner of my brains. Lucky for you, I've dusted it off and brought it to section.

⁵Or $[3,2]$ - the variance-covariance matrix is symmetric. If you don't know why the estimates we care about are coming from these elements of the variance-covariance matrix, think hard about what this matrix looks like and why.

⁶This should look suspiciously familiar.

```

vCov <- function(data, y, X){
  n <- nrow(data)
  k <- length(X)
  ydata <- tblldfGrabber(data, y)
  xdata <- tblldfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  e <- ydata - xdata %*% betahat
  # we have to turn s2 into a vector object to multiply it by XpXinv
  # because it's automatically generated as a 1-by-1 matrix
  # rather than a scalar.
  # R is dumb sometimes.
  s2 <- as.vector((t(e) %*% e) / (n-k))
  XpXinv <- solve(t(xdata) %*% xdata)
  vcov <- s2 * XpXinv
  return(vcov)
}

```

Great, let's try it out:

```

myVCov <- vCov(autos, "price", c("ones", "mpg", "weight"))

varB1 <- myVCov[2,2]
varB2 <- myVCov[3,3]
covB1B2 <- myVCov[2,3]

```

Armed with all of these quantities, we can (finally) compute our analytic standard error:

```

lcSE <- sqrt(((20^2) * varB1) + ((3000^2) * varB2) + (2*20 * 3000 * covB1B2))

```

Now we can actually look at our point estimate and standard error together:

```

LC

## [1] 4249.433

lcSE

## [1] 3467.471

```

Let's make sure that we did our math right by checking against the canned version, using the `estimable()` function from the `gmodels` package.⁷ Unfortunately, the `estimable()` function doesn't work with `felm` objects, only with `lm` objects⁸:

⁷If you're a Stata person, this is the equivalent of `-lincom-`.

⁸The good news is that the next tool we'll use for calculating these standard errors, which we'll do more often anyway, should work just fine. The other good news is that even if you're trying to run a regression using a bunch of fixed effects, you now know how to do Frisch-Waugh-Lovell, so you can do the final step in `lm()`.

```
cannedReg <- lm(data = autos, price ~ mpg + weight)
lcVals <- c(0, 20, 3000)
cannedLC <- estimable(cannedReg, lcVals)

cannedLC

##              Estimate Std. Error  t value DF  Pr(>|t|)
## (0 20 3000) 4249.433    3467.471  1.225514 71  0.2244315
```

Boom! Great: we have an estimate of 4249 with a standard error of 3467. Pop quiz: is this thing statistically significant? Remember that the t statistic is just $\hat{\beta}/\text{se}(\hat{\beta})$:

```
(lctStat <- LC / lcSE)

## [1] 1.225514
```

So this isn't statistically significant at conventional levels, since for an estimate to be statistically significant at 95% confidence requires $t \geq 1.96$.

Delta

If you don't feel like solving for your standard errors analytically, fear not! There's another way. We can implement the delta method. It can be intimidating at first, but is incredibly useful, so bear with me. A couple of examples should help clear it up. Before we get into the math, a quick reminder of what we're trying to get: we want an estimate of, and a standard error on, $LC = 0\alpha + 20\beta_1 + 3000\beta_2$. With that in mind, let's go to the theorem:

Let $\mathbf{A}(\boldsymbol{\beta}) \equiv \frac{\partial \mathbf{a}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}}$. Max's lecture notes tell us:

$$\sqrt{N}(\mathbf{x}_N - \boldsymbol{\beta}) \xrightarrow{d} N(\mathbf{0}, \boldsymbol{\Sigma})$$

This implies:

$$\sqrt{N}(\mathbf{a}(\mathbf{x}_N) - \mathbf{a}(\boldsymbol{\beta})) \xrightarrow{d} N(\mathbf{0}, \mathbf{A}(\boldsymbol{\beta})\boldsymbol{\Sigma}\mathbf{A}(\boldsymbol{\beta})')$$

Hold the phone. Let's define some terms to make this make sense in our world. Let's start with $\mathbf{a}(\cdot) \equiv LC(\cdot)$, and $\mathbf{x}_N = \hat{\boldsymbol{\beta}}$, so $\mathbf{a}(\mathbf{x}_N)$ is just $0\hat{\alpha} + 20\hat{\beta}_1 + 3000\hat{\beta}_2$, and $\mathbf{a}(\boldsymbol{\beta})$ is just $0\alpha + 20\beta_1 + 3000\beta_2$. This also means that we can write $\mathbf{A}(\boldsymbol{\beta}) = \mathbf{LC}(\boldsymbol{\beta}) = [0 \ 20 \ 3000]$.⁹

We're almost out of the woods. We can now write:

$$\sqrt{N}(\hat{\boldsymbol{\beta}} - \boldsymbol{\beta}) \xrightarrow{d} N(0, \boldsymbol{\sigma}^2(\mathbf{X}'\mathbf{X})^{-1})$$

and

$$\sqrt{N}(LC(\hat{\boldsymbol{\beta}}) - LC(\boldsymbol{\beta})) \xrightarrow{d} N(0, \mathbf{LC}(\boldsymbol{\beta})\boldsymbol{\sigma}^2(\mathbf{X}'\mathbf{X})^{-1}\mathbf{LC}(\boldsymbol{\beta})')$$

⁹This is because we defined $\mathbf{A}(\boldsymbol{\beta})$ above to be the derivative of $\mathbf{a}(\boldsymbol{\beta})$ with respect to $\boldsymbol{\beta}$. Too. Much. Notation!

so that means that $V(LC) \approx \mathbf{LC}(\boldsymbol{\beta})\boldsymbol{\sigma}^2(\mathbf{X}'\mathbf{X})^{-1}\mathbf{LC}(\boldsymbol{\beta})'$. Phew - done! That's the entire method.

To sum up, to get the variance of our linear combination of coefficients, LC , all we have to do is pre- and post-multiply $\sigma^2(\mathbf{X}'\mathbf{X})^{-1}$ by the gradient of LC , $\mathbf{LC}$. Like before, the standard error is just the square root of the variance. In code:

```
bigLC <- matrix(c(0, 20, 3000), ncol = 3)
deltaSE <- sqrt(bigLC %*% myVCov %*% t(bigLC))

deltaSE

##           [,1]
## [1,] 3467.471
```

It works! Amazing. I love standing on the shoulders of giants.

2/2 = 5?

We now know how to compute standard errors for linear combinations of coefficients. But sometimes that just won't cut it - like, for instance, if we want to compute standard errors for *nonlinear* combinations of coefficients.¹⁰ Suppose, for example, that we have the following data generating process:

$$y_i = \alpha + \beta_1 x_i + \beta_2 x_i^2 + \varepsilon_i$$

This will generate a quadratic relationship (as long as $\beta_2 \neq 0$, of course), which is fine - OLS can handle it, as can our analytical and delta methods above, if all we wanted to do was calculate the standard error of a linear combination of these coefficients.

But what if we want to do something else? Suppose we'd like to know at what value of x our line is maximized? We can of course use calculus to do this: take the derivative of the above function with respect to x , and if you did it right, you'd get $x^{MAX} = \frac{-\hat{\beta}_1}{2\hat{\beta}_2}$. With an estimate of $\hat{\beta}$, we can calculate $\hat{x}^{MAX}$ easily as $\frac{-\hat{\beta}_1}{2\hat{\beta}_2}$. To do this, let's generate some fake data:

```
e <- rnorm(50)
x <- runif(50, -2, 2)
x2 <- x^2

alpha <- 3
beta <- c(2, -2)

y <- alpha + beta[1]*x + beta[2]*x2 + e

quadData <- cbind(y, 1, x, x2) %>%
  as.tbl_df()
names(quadData) <- c("y", "ones", "x", "x2")
```

¹⁰I'm [shocked, shocked](#), to find that gambling is going on in here! (If you haven't seen this movie, take the night off from econometrics and watch it now.)

Next, compute x^{MAX} and $\hat{x}^{MAX}$:

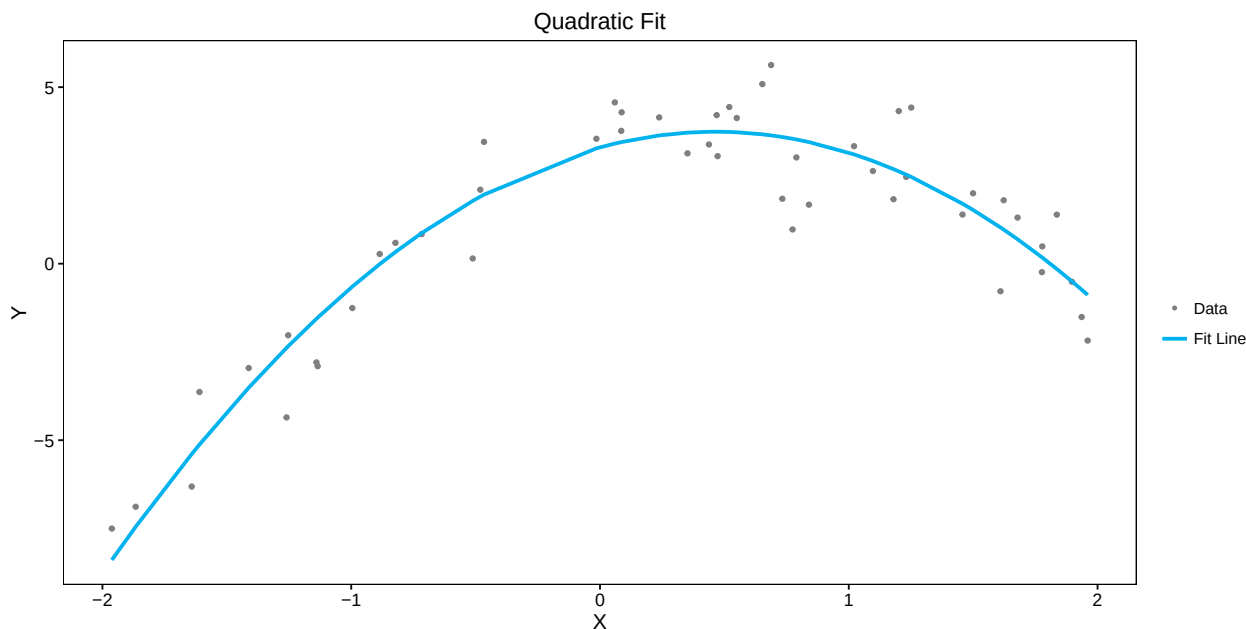
```
xMax <- -beta[1] / (2*beta[2])
quadOLS <- OLS(quadData, "y", c("ones", "x", "x2"))
#note we need the 2nd and 3rd estimated params b/c we dont want the int
xMaxHat <- -(quadOLS$betahat)[2] / (2 * (quadOLS$betahat)[3])
```

We could compare by just looking at the output - but that's dull. Instead, let's compare by plotting our data, our estimated function, the true x^{MAX} , and our estimated $\hat{x}^{MAX}$. We'll start by creating an estimated $\hat{y}$, and plotting the data and the fit line:

```
# create yhat
quadData <- mutate(quadData, yhat = (quadOLS$betahat[1]*ones +
                                     quadOLS$betahat[2]*x + quadOLS$betahat[3]*x2))

myPlot <- ggplot(data = quadData, aes(x = x)) +
  geom_point(aes(y = y, color = "Data"), size = 1) +
  geom_line(aes(y = yhat, color = "Fit Line"), size = 1) +
  myThemeStuff +
  # color the dots & line. guide lets us use the guides command below
  scale_color_manual("", values = c('gray50', 'deepskyblue2'), guide = 'legend') +
  # prevents the Data entry in the legend from having a line
  guides(color = guide_legend(override.aes =
                             list(linetype = c(0, 1), shape = c(16, NA)))) +
  xlab("X") + ylab("Y") + ggtitle("Quadratic Fit")
```

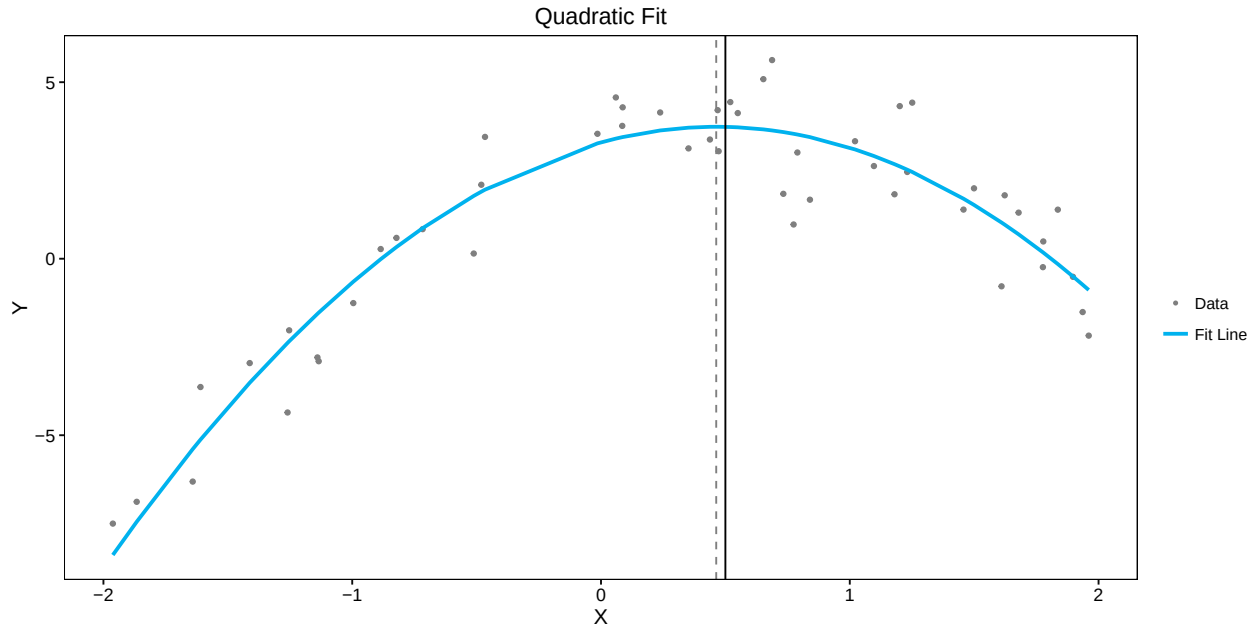
myPlot



Now we can take advantage of **ggplot2**'s additive plotting capabilities and add two vertical lines: one for the true x^{MAX} , and another for our estimated $\hat{x}^{MAX}$:


```
myXmaxPlot <- myPlot +
  geom_vline(xintercept = xMax, color = "black") +
  geom_vline(xintercept = xMaxHat, linetype = "dashed", color = "gray50")
```

```
myXmaxPlot
```



Pretty. Darn. Close.¹¹ Cool!

But let's not stop there. We don't want to just calculate $\hat{x}^{MAX}$ - we want to compute its standard error. Unlike in the `autos` data example, we can't use our simple variance formulas to linearly combine coefficients to get the standard error here. There is an equation to do this, wherein we treat the elements of $\hat{\beta}$ as random variables, and we can calculate the variance of a ratio of random variables - but the formula for this is super gross, so we're going to move on.¹²

Rather than getting bogged down in one particular variance formula, let's turn to our friend the delta method. As we showed above, the delta method is perfectly adequate for calculating standard errors of linear combinations of coefficients - but it's really useful when you're trying to compute standard errors of products, quotients, or other non-linear combinations of coefficients (gulp!).

In this application, we'll let $\mathbf{a}(\cdot) = x^{MAX}(\cdot)$. This lets us write that $\mathbf{a}(\beta) = \frac{-\beta_1}{2\beta_2}$, and $\mathbf{A}(\beta) = \begin{bmatrix} \frac{-1}{2\beta_2} & \frac{\beta_1}{2\beta_2^2} \end{bmatrix}$. As usual, we can write $\mathbb{V}(x^{MAX}) \approx \mathbf{A}(\beta)\sigma^2(\mathbf{X}'\mathbf{X})^{-1}\mathbf{A}(\beta)'$. Beautiful. We will of course estimate this using our estimates of β_1 and β_2 . "Hard" math done, let's code it up:

¹¹I rigged this example to only have 50 observations so that we could see both lines on the graph. Up the number of observations, and clearly $\hat{x}^{MAX}$ will converge quickly to x^{MAX} .

¹²Okay, nerds: go look up "On the Arithmetic Means and Variances of Products and Ratios of Random Variables" by Fred Frishman.

```

# for ease of notation
# note that we can use the semicolon to put two things on the same line
#   for you fancy folks, this is a "delimiter" and tells R
#   to evaluate anything after it as a new line
beta1 <- quadOLS$betahat[2]; beta2 <- quadOLS$betahat[3]
A <- matrix(c(0, -1/(2*beta2), beta1/(2* beta2^2)), nrow = 1)
vcov <- vCov(quadData, "y", c("ones", "x", "x2"))

(mySE <- sqrt(A %*% vcov %*% t(A)))

##           [,1]
## [1,] 0.04020689

```

Let’s quickly make sure that we’re doing this right, using the `msm` package’s `deltamethod()` function:

```

cannedOLS <- felm(data = quadData, y ~ x + x2)
(cannedSE <- deltamethod(~-x2/(2 * x3), coef(cannedOLS), vcov(cannedOLS)))

## [1] 0.04020689

```

Boommmmmmm.

Now that we’ve calculated the standard error, let’s add the confidence intervals to our figure:

```

n <- dim(quadData)[1]
k <- 3
# create CIs by computing estHat +/- t(alpha=0.25, n-k dof) * SE
lowerCI <- xMaxHat - qt(0.975, n-k) * mySE
upperCI <- xMaxHat + qt(0.975, n-k) * mySE

```

And once again, we’ll take advantage of `ggplot2`’s additivity properties:

```

fullPlot <- myXmaxPlot +
  geom_vline(xintercept = lowerCI, linetype = "dotted", color = "gray50") +
  geom_vline(xintercept = upperCI, linetype = "dotted", color = "gray50")

```

Looks like it contains the true value to me! Excellent.

That thing macroeconomists do

Every now and again, economists are asked to make forecasts or predictions.¹³ Since you’re all responsible microeconometricians, you will *of course* provide a measure of uncertainty around your prediction - we call this the “prediction error.” Let’s spend a few minutes thinking about how to compute a prediction and its error.

Our (very general) model is:

$$\mathbf{y} = \mathbf{X}\mathbf{b} + \epsilon$$

¹³As Betty once famously remarked, “Some are better than others.” (For full effect, imagine this in her French accent.)

This makes it very easy for us to generate predictions - in fact, we've basically been doing it already when we've been generating values of $\hat{y}$. This is *in-sample* prediction: using our observed $\mathbf{X}$ data and our estimated $\hat{\beta}$ coefficients to make a prediction about our outcome variable, $\hat{y}$. We're often interested in *out-of-sample* prediction, which is just like in-sample prediction, except instead of combining our $\hat{\beta}$ estimates with observed $\mathbf{X}$ data, we combine it with whatever $\mathbf{X}$ data we choose.¹⁴ Let's think about making a prediction for the data series $y^0, \mathbf{X}^0$.

There are actually two additional interesting quantities to calculate here: the confidence interval around the prediction, and the prediction interval. The confidence interval tells us the confidence we should have in our estimate of $\hat{y}^0 = \mathbf{X}^0 \hat{\beta}$, and the prediction interval tells us the error we should expect when we predict new values of y . Notice that this contains both sampling error and true error (ϵ) from the DGP - if we were to have a perfect estimate of β , our confidence interval would converge to include only our point estimate, but our prediction interval wouldn't. This means that prediction intervals will always be (weakly) larger than confidence intervals.

In math, the standard error of the mean prediction (which we'll use to generate the confidence interval) is: $\sqrt{\mathbb{V}(\hat{\mathbf{y}}^0)}$, and the prediction error (which we'll use to generate the prediction interval) is $\sqrt{\mathbb{V}(\hat{\mathbf{y}}^0 - \mathbf{y}^0)}$. Notice that we can derive a formula for the variance component of each of these objects (I'll leave off the square root bit for now because it's tedious to type repeatedly):

Variance of the prediction:

$$\begin{aligned}\mathbb{V}(\hat{\mathbf{y}}^0) &= \mathbb{V}(\mathbf{X}^0 \hat{\beta}) \\ &= \sigma^2 (\mathbf{X}^0 (\mathbf{X}' \mathbf{X})^{-1} \mathbf{X}'^0)\end{aligned}$$

Variance of the prediction error:

$$\begin{aligned}\mathbb{V}(\hat{\mathbf{y}}^0 - \mathbf{y}^0) &= \mathbb{V}(\mathbf{X}^0 \hat{\beta} - \mathbf{X}^0 \beta - \epsilon^0) \\ &= \sigma^2 (1 + \mathbf{X}^0 (\mathbf{X}' \mathbf{X})^{-1} \mathbf{X}'^0)\end{aligned}$$

I'm skipping the math in the interest of time, but it's nothing crazy - we just use our variance rules from above in conjunction with our OLS assumptions and middle school algebra.¹⁵ We could also use the delta method to get to the same endpoint - to do this, notice that $\mathbb{V}(\hat{\mathbf{y}}^0 - \mathbf{y}^0) = \sigma^2 + \mathbb{V}(\mathbf{X}^0 \hat{\beta})$. Use the delta method to compute $\mathbb{V}(\mathbf{X}^0 \hat{\beta})$, letting $\mathbf{a}(\hat{\beta}) = \mathbf{X}^0 \hat{\beta}$, and $\mathbf{A}(\hat{\beta}) = \mathbf{X}^0$. This will get us to the same answer as above.

We'll use one of `ggplot2`'s built-in datasets, `diamonds`, to explore these ideas.¹⁶ We'll start by taking a random sample of this dataset (so that our prediction interval computations won't take so long):

```
# grab the dataset
diamondData <- diamonds %>%
  # add a column of ones, of course
  mutate(ones = 1) %>%
  sample_n(500, replace = F)
```

¹⁴A simple example: say we want to predict GDP over time. Our X data are years. Our Y data are GDP observations. Our time series goes from 1950-2016. To do an in-sample prediction, we'd run our regression of GDP on year, and then use our estimated $\hat{\beta}$ to make predictions using years in the data. To do an out-of-sample prediction, we'd use the same estimated $\hat{\beta}$ prediction with years in a range like $year \in 2020, 2030$. One downside of this: it's hard to tell how well your model is doing!

¹⁵Holllaaaaaa! If you want to see a derivation, click [here](#).

¹⁶A new dataset! I told you it'd happen eventually.

Let's run a regression of price¹⁷ on carats, and save the predicted values:

```
diamOLS <- OLS(diamondData, "price", c("ones", "carat"))
diamondData <- mutate(diamondData, priceHat = diamOLS$betahat[1] + carat * diamOLS$betahat[2])
```

Now let's actually compute the errors that we're interested in. Notice that when we computed a standard error on $\hat{\beta}$ before, we computed one standard error for each coefficient. Now we're predicting N values for $\hat{y}$, so we should get N standard errors. We're going to actually *also* return the confidence interval and the prediction interval.

Let's get to codin':

```
# we use newdata to tell our fxn what x's we'd like make (yhat) predictions about
predictError <- function(data, y, X, newdata){
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  # in-sample prediction
  yhat <- xdata %*% betahat
  e <- ydata - yhat
  s2 <- as.vector((t(e) %*% e) / (n-k))
  XpXinv <- solve(t(xdata) %*% xdata)

  # out of sample prediction
  newx <- as.matrix(newdata)
  newy <- as.vector(newx %*% betahat)

  #confidence error
  ce <- diag(sqrt(s2 * (newx %*% XpXinv %*% t(newx))))
  #prediction error
  pe <- diag(sqrt(s2 * (1 + newx %*% XpXinv %*% t(newx))))
  confIntLower <- newy - qt(0.975, n-k) * ce
  confIntUpper <- newy + qt(0.975, n-k) * ce
  confInt <- cbind(confIntLower, confIntUpper)
  # PI
  predIntLower <- newy - qt(0.975, n-k) * pe
  predIntUpper <- newy + qt(0.975, n-k) * pe
  predInt <- cbind(predIntLower, predIntUpper)

  output <- list(newy, ce, pe, confInt, predInt)
  names(output) <- c("yhat", "CE", "PE", "confInt", "predInt")
  return(output)
}
```

¹⁷Okay, so it's a new dataset - but not a new variable...

Let's create a new $\mathbf{X}^0$ dataset to use as our prediction. We'll predict price for carats between 0.5 and 10, in steps of 0.5. This will be both some in-sample predictions, as well as some out-of-sample predictions, since our data goes from 0.2 carats to 4.1 carats.¹⁸

```
# create a new dataset of carats from 0.5 to 10, in steps of 0.5
predictionData <- seq(0.5, 10, 0.5) %>%
  # add a col of ones to it
  cbind(1, .) %>%
  # convert to tbl_df
  as.tbl_df()
# name it appropriately
names(predictionData) <- c('ones', 'carat')
```

And let's run our function¹⁹:

```
diamError <- predictError(diamondData, "price", c("ones", "carat"), predictionData)
```

Let's quickly check our results against the canned routine (again, you'll have to use `lm()` rather than `felm()` for this). One (annoying) thing to note is that the canned routine won't spit out the standard error it uses to generate prediction intervals. It will give you the standard error of the mean prediction, and the upper and lower intervals - so you could back it out yourself, but ain't nobody got time for that (at least not in section). But we'll know if our routine worked correctly if the confidence intervals are right, so we'll use those to compare. First, create the canned predictions

```
cannedLM <- lm(data = diamondData, price ~ carat)
cannedYhat <- predict(cannedLM, predictionData)
# se.fit says grab SE's, interval suggests the interval type, and lfit grabs the CI
cannedCI <- (predict(cannedLM, predictionData, se.fit = T, interval = "confidence"))$fit
cannedPI <- (predict(cannedLM, predictionData, se.fit = T, interval = "prediction"))$fit
```

And compare:

```
all.equal(diamError$yhat, cannedYhat, check.attributes = FALSE)
## [1] TRUE

all.equal(diamError$confInt, cannedCI[, -1], check.attributes = FALSE)
## [1] TRUE

all.equal(diamError$predInt, cannedPI[, -1], check.attributes = FALSE)
## [1] TRUE
```

Looking good! As a final exercise, let's confirm our intuition that the confidence interval should always be weakly smaller than the prediction interval by (you guessed it) plotting them. As usual, put our plot stuff into a dataframe:

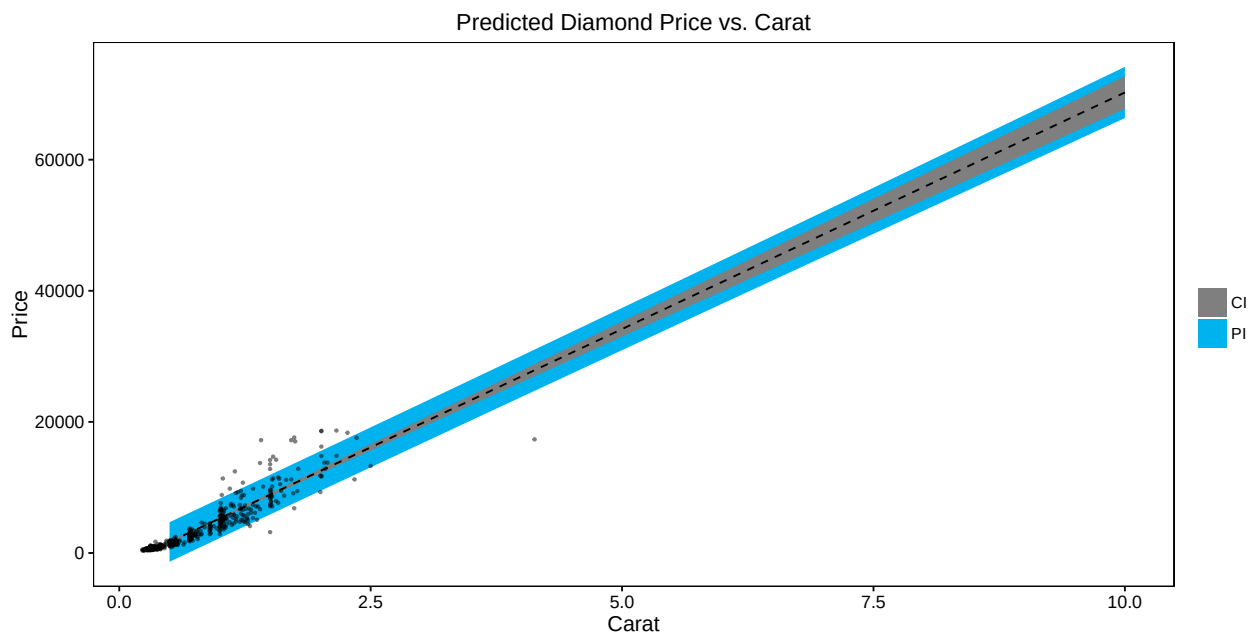
¹⁸Who wants a 10 carat diamond? That thing would weigh you down like nobody's business.

¹⁹This might create a warning for you - don't worry about it - the elements on the diagonal will not be NaNs. This is just worried about square roots of negative numbers, which won't end up on the diagonal.

```
diamondDF <- as.tbl_df(cbind(diamondData$carat, predictionData[,-1], diamondData$price,
                           diamError$yhat, diamError$confInt, diamError$predInt))
names(diamondDF) <- c("caratreal", names(predictionData[,-1]), "y",
                     "yhat", "ciLow", "ciHigh", "piLow", "piHigh")
```

```
myDiamondPlot <- ggplot(data = diamondDF) +
  #PI
  geom_ribbon(aes(x = carat, ymin = piLow, ymax = piHigh, fill = "PI")) +
  #CI
  geom_ribbon(aes(x = carat, ymin = ciLow, ymax = ciHigh, fill = "CI")) +
  # real data
  geom_point(aes(x = caratreal, y = y), alpha = 0.5, size = 0.5, color = "black") +
  # predicted line
  geom_line(aes(x = carat, y = yhat), size = 0.5, color = "black", linetype = "dashed") +
  scale_fill_manual(values = c("gray50", "deepskyblue2")) +
  myThemeStuff +
  xlab("Carat") + ylab("Price") + ggtitle("Predicted Diamond Price vs. Carat") +
  theme(legend.title = element_blank())
```

myDiamondPlot



Beautiful. Notice that the confidence interval is quite small in the support of our data, while the prediction interval remains wide. The confidence interval widens away from our support. We'll leave things here for now, and pick up next week with more non-standard standard errors.